Техническое задание: Разработка сервиса управления заказами

1. Введение

Разработать сервис управления заказами на **FastAPI**, поддерживающий аутентификацию, работу с очередями сообщений, кеширование и фоновую обработку задач.

2. Функциональные требования

2.1 API эндпоинты

Метод	URL	Описание
POST	`/register/`	Регистрация пользователя (email, пароль)
POST	`/token/`	Получение JWT-токена (OAuth2)
POST	`/orders/`	Создание заказа (Только авторизованные)
GET	`/orders/{order_id}/`	Получение заказа (сначала из Redis)
PATCH	`/orders/{order_id}/`	Обновление статуса заказа
GET	`/orders/user/{user_id}/`	Получение заказов пользователя

2.2 База данных (PostgreSQL)

Таблица **orders**:

- `id` (UUID, primary key)
- `user_id` (int, ForeignKey на пользователей)
- `items` (JSON, список товаров)
- `total_price` (float)
- `status` (enum: PENDING, PAID, SHIPPED, CANCELED)
- `created_at` (datetime)

2.3 Очереди сообщений (Kafka / RabbitMQ)

- При создании заказа публиковать событие `"new_order"` в очередь.
- Консьюмер обрабатывает заказ и передает его в Celery.

2.4 Redis (Кеширование заказов)

- Если заказ запрашивается повторно – **отдавать его из кеша** (TTL = 5 минут).
- При изменении заказа – **обновлять кеш**.

2.5 Celery/taskiq (Фоновая обработка)

- Фоновая задача **обработки заказа** (`time.sleep(2)` и `print(f"Order {order_id} processed")`).

2.6 Безопасность

- **JWT-аутентификация** (OAuth2 Password Flow).
- **CORS-защита** (ограничение кросс-доменных запросов).
- **Rate limiting** (ограничение частоты запросов на API).
- **SQL-инъекции** – только ORM-запросы.

3. Нефункциональные требования

- Использование **FastAPI** с **Pydantic**.
- Работа с PostgreSQL через **SQLAlchemy** + **Alembic**.
- Асинхронное взаимодействие с **Kafka / RabbitMQ**.
- **Docker Compose** для развертывания всей инфраструктуры.
- **Код должен быть структурированным и документированным**.

4. Инструкция по сдаче

1. **Разместить код на GitHub / GitLab**.
2. **Описать установку и запуск в README.md**.
3. **Прислать ссылку на репозиторий**.
4. **Обязателен SwaggerUI**

5. Критерии приёмки:

– работоспособность всех API, описанных в ТЗ

– корректная авторизация и защита эндпоинтов

– реальная работа Redis, брокера сообщений и фоновых задач

– возможность развернуть проект через docker-compose

– наличие и актуальность Swagger и README.md

– соответствие реализации бизнес-сценариям, описанным в ТЗ

– корректная обработка ошибок и возврат соответствующих HTTP-статусов

– соблюдение заявленного технологического стека

– отсутствие хардкода чувствительных данных и конфигураций